

RFC-0009

Implementação da Camada Web / REST Endpoints e Integração de Segurança

Status: Aprovado e Atualizado (v1.1)

Sistema: API de Serviços SEAGRI-DF

Plataforma: Spring Boot / Java

Segurança: Keycloak + Spring Security Resource Server

Banco de dados: PostgreSQL

Modelo: Multitenant com Schemas Segregados

Depende de: RFC-0001, RFC-0005, RFC-0008, Arquitetura de Identidade e Acesso Segura

1. Objetivo

Definir os padrões técnicos para a implementação da camada Web (REST Controllers) da aplicação.

Essa camada será responsável por:

- Expor endpoints HTTP de forma padronizada.
- Validar requisições na borda (Jakarta Validation).
- Delegar chamadas para a camada de aplicação de forma limpa.
- Padronizar respostas da API (JSON único).
- Integrar segurança de forma agnóstica às regras de IAM (Isolamento de Domínio).

A implementação deve respeitar as decisões arquiteturais já definidas nas RFCs anteriores (DDD, Ports & Adapters, Comando Único e Identidade Canônica Centralizada). A camada Web deve permanecer extremamente fina, contendo apenas responsabilidades de interface e transporte de DTOs.

2. Contexto do Sistema

A API possui as seguintes características inegociáveis de segurança:

- **Autenticação via Keycloak:** A API não valida senhas, ela consome *Lightweight Access Tokens* e confia na assinatura criptográfica (OIDC).
- **Modelo Multitenant:** As regras de acesso são ditadas pelo `organizacao_id` e avaliadas dinamicamente.
- **Confiança Zero (Zero Trust):** A camada Web não confia nos dados enviados pelo frontend (React), extraíndo a identidade do usuário através de integrações profundas e interceptadores

do Spring Security.

3. Estrutura da Camada Web

A camada Web utilizará uma estrutura baseada em *controllers* abstratos reutilizáveis, permitindo a padronização e a redução drástica de duplicação de código (*boilerplate*).

4. Controller Base da API e Extração de Identidade

Será criado um controller base genérico, responsável pelos comportamentos comuns a toda a camada Web.

Componente: `AbstractApiController`

Esse controller não assume operações CRUD e oferecerá funcionalidades centrais, integrando-se estritamente à segurança do Spring:

- **Extração da Identidade Canônica:** Em vez de extrair a `claim sub` diretamente do JWT, este controller acessará o `SecurityContextHolder` para resgatar o **`BusinessPrincipalToken`** (injetado precocemente pelo `customJwtAuthenticationConverter`). Isso garante que os *endpoints* já saibam o CPF da pessoa física e suas *Roles* locais aplicáveis.
- **Acesso ao Tenant:** Acesso facilitado ao identificador da organização atual (repassado via requisição ou rota).
- **Envelopamento:** Utilitários para construção do envelopamento padronizado de respostas (`ApiResponse`).

Todos os controllers da aplicação estenderão direta ou indiretamente este componente.

5. Controllers Especializados para CRUD (Dupla Hierarquia)

Para entidades que expõem operações padrão de persistência, utilizaremos uma dupla hierarquia de controllers genéricos, alinhada à nossa camada de serviços e ao isolamento de *Schemas*.

5.1. Contexto Global (Sem Tenant)

Para tabelas de apoio e domínios globais que residem no schema `pessoa` (ex: Profissões, Nacionalidades). **Componente:** `AbstractCrudController`

5.2. Contexto Isolado (Multi-Tenant)

Para as entidades exclusivas de cada organização dentro do schema `folha_ponto`. Este controller injetará compulsoriamente a validação de segurança de fronteira. **Componente:** `AbstractCrudTenantController`

Ambos os controllers deverão:

- Expor endpoints CRUD padrão (GET, POST, PUT/PATCH, DELETE).
 - Delegar operações aos respectivos Application Services (CrudSrv ou CrudTenantSrv).
 - Forçar a injeção do parâmetro `organizacao_id` em todas as rotas do contexto isolado para o enforcement de multitenancy.
-

6. Tratamento de Entidades Imutáveis (Comando Único)

Em conformidade com as RFC-0005 e RFC-0008 (Comando Único e Imutabilidade), entidades estritamente imutáveis **jamais deverão estender os controllers genéricos de CRUD**.

Exemplo: `RegistroPresencaController` Este controller deverá estender diretamente o `AbstractApiController`, expondo **exclusivamente**:

- Rotas de **POST** (para recepção do evento bruto com GPS e Foto destinada à quarentena visual temporária com expurgo agendado).
- Rotas de **GET** (para listagem de histórico, auditoria e relatórios).

Sob nenhuma hipótese rotas de PUT ou DELETE serão expostas para essas entidades operacionais críticas.

7. Consultas Simplificadas (Lookup para UI)

O sistema precisará alimentar componentes de interface de usuário (React SPA) como `select`, `combobox` e `autocomplete` através de listas simplificadas e não paginadas.

Os controllers abstratos de CRUD exporão um endpoint de lookup de forma nativa. **Exemplo de rota embutida:** `GET /api/v1/organizacoes/{orgId}/unidades/lookup`

Esse endpoint retornará a coleção de DTOs mínimos (Apenas `id` e `nome`), melhorando o desempenho da rede.

8. Padronização de Respostas da API

Todas as respostas HTTP do Spring Boot devem seguir um formato consistente e previsível.

Componente: `ApiResponse<T>`

Campos obrigatórios:

- `timestamp`: Horário oficial do servidor em UTC.
 - `status`: Código numérico HTTP.
 - `message`: Mensagem descritiva (ex: "Registro salvo com sucesso").
 - `payload`: Objeto T ou Lista `List<T>` contendo os dados.
 - `errors`: Lista de detalhes de validação ou restrição.
-

9. Tratamento Global de Exceções e Respostas SPA-Friendly

A API possuirá um tratamento centralizado de erros (via `@ControllerAdvice`). Para que o frontend React possa orquestrar rotas de forma limpa, as falhas de segurança do IAM não devem renderizar páginas HTML nativas do Spring.

O *Global Exception Handler* garantirá a devolução do modelo `ApiResponse` para:

- **Erros de Segurança (401/403):** Utilizando implementações customizadas no Resource Server (`customAccessDeniedHandler` e `customAuthenticationEntryPoint`), erros de token expirado ou violação de Tenant retornarão JSON formatado, nunca um *redirect* HTML.
 - **Erros de Validação:** Interceptação de falhas nos DTOs.
 - **Erros de Negócio:** Violações lançadas pela camada de *Domain/Application*.
 - **Erros do PBAC:** O *Policy Decision Point* retornará uma negativa estruturada com o motivo da rejeição.
-

10. Validação Automática de Requisições

As requisições utilizarão validação declarativa baseada no *Jakarta Validation* (`@Valid`, `@NotNull`, `@NotBlank`, `@Size`). A camada Web bloqueia dados malformados antes de invocarem a regra de negócio.

11. Documentação e Versionamento

- **OpenAPI (Swagger):** Documentação gerada automaticamente para os contratos da API.
 - **Versionamento:** Todos os endpoints utilizarão versionamento semântico nas URLs (`/api/v1/...`) para assegurar retrocompatibilidade.
-

12. DTOs, Mapeamento e a Diretriz "Fetch On-Demand" (LGPD)

O trânsito de dados através da camada REST é o principal vetor de vazamentos (Data Breach). A aplicação consumirá e retornará exclusivamente DTOs (via **MapStruct**).

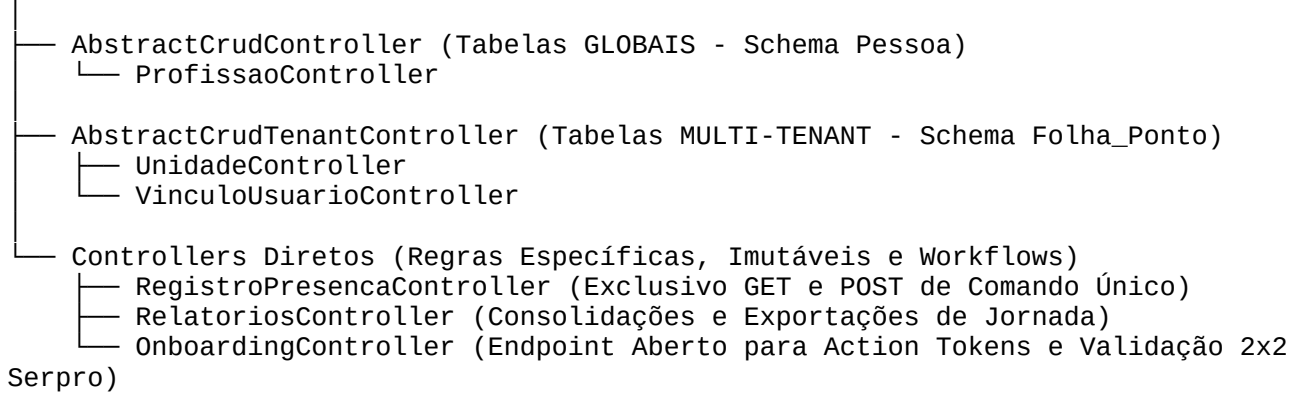
Para cumprir a conformidade exigida pelo documento de **Arquitetura de Identidade Segura**:

- **Fetch On-Demand Obrigatório:** Endpoints de listagem genérica (`GET /usuarios`) usarão DTOs magros, excluindo colunas sensíveis da tabela `pessoa_fisica` e `contato_endereco`.
- **Blindagem de Dados Sigilosos:** CPF, Endereço de geolocalização e Hashes Biométricos nunca poderão vazar nas listagens padrão. Eles só serão trafegados em DTOs dedicados de endpoints específicos de segurança máxima (ex: `GET /api/v1/users/me`), acessados sob demanda pelo React SPA.

13. Estrutura Conceitual Final da Camada Web

A organização dos *Controllers* obedecerá a esta hierarquia arquitetural definitiva:

AbstractApiController (Base com Identidade de Negócio e ApiResponse)



Nota sobre o OnboardingController: Este é o controlador especializado projetado para interceptar a requisição de completude de perfil (Foto + CPF). Ele estará configurado para validar não o *Access Token* principal, mas o JWT de curta duração (*Action Token*) emitido pelo SPI do Keycloak durante a pausa de login inicial do usuário.
